An Empirically Verified Fixed-Point Stable Compiler for the spaces Programming Language: Achieving Provenance Closure in the Void

Abstract

The relentless proliferation of visible syntax in modern programming languages has precipitated an unprecedented level of visual pollution, cognitive overload, and aesthetic degradation within the broader software engineering discipline. In response to this escalating crisis, the overarching field of computational heresy has long sought a paradigm wherein the source code itself becomes an invisible, silent substrate. This paper presents an exhaustive, mathematically rigorous analysis of spaces, a novel esoteric programming language strictly constructed from semi-significant whitespace. Specifically, the language capitalizes on the bipartite topological distinction between the half-width space (U+0020) and the full-width space (U+3000) to encode Turing-complete logic. Beyond mere syntactic novelty or typographic obfuscation, the spaces language achieves a monumental milestone in theoretical computer science: a cryptographically verified, self-hosting fixed-point compiler demonstrating absolute provenance closure. By meticulously tracing the compiler's bootstrap process from a primitive C-based Virtual Machine to a standalone Native Linux Executable and Linkable Format (ELF) binary generated via deeply nested, meta-programmed macro-architectures, this investigation proves that spaces is capable of pure, unadulterated self-replication. This report further examines the intricate lexical semantics of the language, the computational complexities of emitting binary executable headers directly from the void, and the rigorous Continuous Integration protocols utilized to verify the compiler's stability. Furthermore, addressing the inherent paradox of analyzing invisible phenomena, this report introduces a revolutionary optical visualization technique. As explicitly noted for the reader's convenience, the dot-matrix image appended in the final materials section at the conclusion of this document is a direct geometric visualization of the compiler.spaces source code, rendering the invisible architecture manifest through a binary mapping of white and black pixels.

1. Introduction

The historical trajectory of programming language design has been fundamentally compromised by a universal, yet entirely arbitrary, reliance on visible glyphs, alphanumeric characters, and explicit punctuation. From the early days of raw assembly language programming to the development of modern high-level paradigms such as Python and visual environments like Scratch , the epistemological assumption has universally been that source code must be physically seen by the human ocular system in order to be understood, maintained, and executed. However, as the broader field of computer science matures under the auspices of the Association for Computational Heresy , researchers have increasingly questioned this deeply ingrained axiom.

Recent empirical investigations have revealed the inherent absurdity of strictly visible syntax. If, as established in prior landmark literature, exactly 93% of random paint splatters on a wall can be successfully parsed as valid Perl programs by optical character recognition software , the semantic value of visual code structure is immediately called into question. Furthermore, if optimal coding productivity is firmly linked not to language syntax, but rather to the precise physiological modulation of blood alcohol content (the empirically verified Ballmer Peak) , or the deployment of highly specific physical garments such as Programmer Socks and the presence of Swedish plush sharks (Blåhaj) , then the physical and visual manifestation of code on a monitor is clearly an arbitrary artifact of mid-20th-century terminal constraints.

The spaces programming language represents the absolute zenith of this intellectual and heretical movement. It operates on a principle of uncompromising visual asceticism. A valid spaces program appears to the untrained eye—and indeed, to the vast majority of conventional text editors, integrated development environments, and version control systems—as a completely blank document, a sprawling void of nothingness. Yet, hidden within this apparent emptiness lies a rigorous, Turing-complete mathematical architecture capable of bootstrapping a fully operational, self-hosting native compiler.

This comprehensive report provides an exhaustive examination of the spaces language ecosystem. It will delineate the precise execution environment, demystify the multi-stage self-compiling bootstrap sequence, and present the cryptographic proof of its mathematical stability. It will be empirically demonstrated that spaces not only functions as a theoretically pure esoteric language but also successfully solves the long-standing "Trusting Trust" problem of compiler provenance. By achieving a verifiable cryptographic fixed point across multiple generations of self-compilation, spaces guarantees its own integrity. Finally, this document will guide the reader toward the supplemental visualization materials located at the end of this report, wherein the invisible source code has been transmuted into a high-density black-and-white dot matrix, revealing the breathtaking structural complexity hidden within the silent void.

2. Epistemological and Historical Context of Computational Heresy

The academic pursuit of invisible, semi-invisible, or highly obfuscated programming constructs is a rich and storied tradition within the annals of the Special Interest Group on Harry Q. Bovik (SIGBOVIK) and the Association for Computational Heresy. To truly comprehend the magnitude of the spaces compiler, one must first situate it within the historical context of its esoteric predecessors and the philosophical frameworks that govern this specific quadrant of academic research.

2.1 The Legacy of Whitespace and Semantic Voids

The seminal Whitespace programming language laid the foundational groundwork for the utilization of non-printing characters as semantic tokens. By strictly utilizing spaces, horizontal tabs, and line feeds, Whitespace constructed a robust stack-based execution environment that entirely ignored visible ASCII characters. This allowed developers to embed functional logic seamlessly within the formatting of standard text files, birthing a new era of steganographic programming. Subsequent theoretical research expanded upon this paradigm, famously attempting to introduce a formal "Type-and-Affect System for Semisignificant Whitespace,"

mathematically proving that the absolute absence of visible text could be rigidly typed, categorized, and subjected to static analysis.
However, despite their groundbreaking nature, previous attempts at whitespace-driven computation suffered from multi-character dependency and structural leakage. By requiring horizontal tabs and line breaks to delineate instructions, these languages inherently disrupted the horizontal flow of the text. The inclusion of line feeds introduced explicit verticality, creating structural anomalies and ragged right margins that immediately betrayed the presence of embedded, artificial logic to any passing observer.
The spaces language forcefully rejects this compromise. It eschews tabs and line feeds entirely, aggressively reducing the lexical universe to exactly two distinct, horizontally contiguous characters: the standard half-width space and the typographic full-width space. This mono-lexical approach mirrors the theoretical frameworks previously discussed in the context of "Pikachu, Domosaur, and Other Monolexical Languages" , but elevates the concept to its ultimate logical conclusion by utilizing purely non-printing characters, thereby achieving true, mathematically perfect visual stealth.

2.2 The Quadrants of Heretical Research

The Association for Computational Heresy formally categorizes its research into three neglected quadrants: joke realizations of joke ideas, joke realizations of serious ideas, and serious realizations of joke ideas. The spaces compiler unequivocally resides in the third quadrant. The premise of a programming language consisting entirely of blank spaces is, on its surface, a humorous concept designed to provoke incredulity. However, the realization of this concept—the engineering of a custom C-based virtual machine, the meta-programming of a Python-based Brainfuck macro assembler, and the mathematical proof of cryptographic provenance closure via automated continuous integration —represents a staggering level of serious, rigorous software engineering.
This juxtaposition of an absurd premise executed with uncompromising academic and technical rigor is the defining hallmark of the highest caliber SIGBOVIK publications. Just as early researchers analyzed the optimal sublinear-time multiplication of very certain, very sparse matrices, or formalized the exact determination of epsilon , the analysis of spaces pushes the boundaries of what is computationally possible when standard engineering constraints are deliberately and maliciously inverted.

3. The Lexical and Semantic Architecture of spaces

The foundational brilliance of spaces lies in its draconian restriction of the input alphabet and its deeply efficient mapping of that restricted alphabet to a Turing-complete instruction set. The lexical analyzer recognizes only two discrete spatial tokens, mapping them directly to binary states. All other characters—whether they be visible letters, numbers, or standard punctuation—are aggressively and silently ignored by the parsing engine, treated as meaningless noise within the vacuum.

3.1 Lexical Tokens and UTF-8 Encoding Sequences

The spaces language differentiates its two foundational tokens not by their visual weight, but by their underlying UTF-8 byte representations. The interpreter requires direct access to the raw

byte stream of the source file to make these distinctions.
The primary token is the **Half-Width Space (U+0020)**. In standard UTF-8 encoding, this character is represented by a single byte: 0x20. Within the semantic architecture of the spaces language, the half-width space is unequivocally mapped to the binary digit 0.
The secondary token is the **Full-Width Space (U+3000)**, commonly utilized in East Asian typography to align with full-width ideographic characters. In UTF-8 encoding, this character spans three distinct bytes, represented sequentially as 0xE3, 0x80, and 0x80. Within the semantic architecture of the spaces language, the full-width space is mapped to the binary digit 1.
This fundamental mapping establishes a continuous, visually homogenous stream of blank text that structurally encodes a dense binary stream. To translate this continuous binary stream into executable, Turing-complete operations, spaces employs a sophisticated 3-bit octal grouping mechanism.

3.2 The 3-Bit Semantic Isomorphism

The spaces language is fundamentally an isomorphic mapping of the classic Brainfuck esoteric programming language. Brainfuck, traditionally dependent on eight highly visible ASCII characters, is entirely subsumed into the void. The spaces parser consumes the binary bits in strict, sequential chunks of three. These three bits are utilized to construct an octal integer V, constrained such that 0 \le V \le 7.
This evaluated integer is then dynamically mapped to the corresponding Brainfuck instruction according to a strictly defined constant array embedded within the compiler architecture: op_map = {'>', '<', '+', '-', '.', ',', '[', ']'}. The exact mapping logic, as executed by the transpilation and parsing layers, is formalized in the following reference table:

3-Bit Binary Sequence	Octal Value (V)	Spatial Encoding (Half/Full Width)	Mapped Operation	Architectural Functionality
000	0	Half, Half, Half	>	Increment the data memory pointer
001	1	Half, Half, Full	<	Decrement the data memory pointer
010	2	Half, Full, Half	+	Increment the byte value at the pointer
011	3	Half, Full, Full	-	Decrement the byte value at the pointer
100	4	Full, Half, Half	.	Output the byte value at the pointer
101	5	Full, Half, Full	,	Accept one byte of standard input
110	6	Full, Full, Half	[	Jump forward past matching] if byte is zero
111	7	Full, Full, Full	]	Jump backward to

3-Bit Binary Sequence	Octal Value (V)	Spatial Encoding (Half/Full Width)	Mapped Operation	Architectural Functionality
				matching The core extraction logic isolates the bits via the formulas: $b_2 = (v \gg 2) \mathbin{\&} 1$, $b_1 = (v \gg 1) \mathbin{\&} 1$, and $b_0 = v \mathbin{\&} 1$. By enforcing this strict, immutable 3-to-1 mapping, thespaces` language ensures that any standard, highly complex algorithm can be perfectly obfuscated into an invisible format, maintaining exact algorithmic parity while completely eradicating code readability and optical comprehension.

4. The Virtual Machine Implementation: Bootstrapping the Void

To execute a program composed entirely of empty space, an initial execution environment must be forged in a traditional, visible language to solve the fundamental chicken-and-egg problem of compiler bootstrapping. To this end, a highly specialized, memory-safe Virtual Machine (VM) was constructed in the C programming language (src/ref/vm.c). This VM acts as the foundational substrate, the Stage 0 execution environment that detects the invisible syntax and breathes life into the empty void.

4.1 Memory Architecture and Rigorous Bounds Checking

The VM memory architecture is instantiated as a contiguous, one-dimensional byte array designated as the tape. Defying standard esoteric language constraints, which often limit tape sizes to a meager 30,000 bytes, the spaces VM tape size is deliberately expanded to a luxurious 65,536 bytes (64 \text{ KB}). This massive expansion is not merely for convenience; it

is a strict requirement, enabling the execution of highly complex macro-structures, such as the full self-hosting compiler, which requires extensive temporary storage and buffer space during the generation of Linux ELF headers.
Given the volatile nature of executing deeply obfuscated code, robust bounds checking is implemented at the core of the instruction loop to prevent spatial anomalies and catastrophic segmentation faults. If the data pointer ptr attempts to transcend the rightward limit (ptr \ge 65536) or underflow the leftward limit (ptr < 0), the VM instantly invokes a critical panic("Tape pointer overflow") function. This function immediately halts the execution context, dumping a standard error message to the console to prevent silent memory corruption and maintain the strict integrity of the execution environment.

4.2 The Lexical Stream Processor and UTF-8 Boundary Enforcement

The primary lexical parsing function, parse_line, is a marvel of resilient byte-stream analysis. It iteratively traverses the raw input buffer, actively suppressing all standard ASCII characters while relentlessly searching for the specific byte signatures 0x20 and 0xE3.
The complexity of parsing UTF-8 in a raw byte array necessitates a sophisticated lookahead mechanism. When the 0xE3 byte is encountered, the VM halts standard processing and delegates authority to the is_full_space routine. This function performs a rigorous spatial boundary check, verifying the contiguous presence of the trailing 0x80 0x80 bytes before registering a logical 1. The explicit C implementation is as follows:

```
int is_full_space(const unsigned char *s, int idx, int len) {
    if (idx + 2 >= len) return 0;
    if (s[idx] == 0xE3 && s[idx+1] == 0x80 && s[idx+2] == 0x80) {
        return 1;
    }
    return 0;
}
```

This strict bounds checking ensures that the parser is entirely immune to malformed UTF-8 sequences or adversarial byte injection. If the boundary check passes, the parser manually increments the index pointer by 2 (i += 2;) to elegantly skip the consumed continuation bytes. The bits are then continuously accumulated in a temporary buffer (bit_buf). Upon reaching a sequence count of three (bit_cnt == 3), the octal value is evaluated, mapped via the op_map array to its ASCII equivalent, and appended to a secondary execution buffer for immediate interpretation.

4.3 Interactive, Piped, and Delegated Execution Paradigms

The Virtual Machine is engineered to support multiple modes of engagement, dynamically adapting to the presence of interactive terminals or automated pipelines. When an interactive TTY session is detected via the POSIX isatty(fileno(stdin)) system call, the VM launches a Safe Mode Read-Eval-Print Loop (REPL). This interactive mode allows researchers to manually inject invisible code sequences via keyboard inputs and observe the immediate memory mutations in real-time.
However, its primary and most vital mode is the processing of massive piped inputs, specifically the 119 KB compiler.spaces file. When a file path is provided as an argument, the VM utilizes fseek to determine the exact binary weight of the file, allocating memory dynamically. A strict 1

\text{ Megabyte} MAX_FILE_SIZE limit is enforced to prevent intentional memory exhaustion attacks by unbounded, infinite emptiness.
Crucially, during the execution of a file, the VM does *not* immediately consume stdin for data input. Instead, it parses the compiler code, executes the resulting instructions via run_bf(), and forces the *compiled compiler* to read its own subsequent input from the intact stdin stream. This precise, delegated input consumption is the mechanical keystone that enables self-hosting pipelines; without it, the VM would consume the source code meant for the compiler, resulting in immediate programmatic starvation.

5. Meta-Compilation: The Python DSL Macro Architecture

The spaces ecosystem does not merely interpret code; its primary objective is to compile invisible source code directly into native, standalone Linux Executable and Linkable Format (ELF) binaries. The generation of this staggering compiler is orchestrated by a deeply nested, meta-programmed Python script: tools/gen_spaces_compiler_bf_bundler.py. This script utilizes an advanced Domain Specific Language (DSL) embedded within Python strings to programmatically generate the immense volume of raw Brainfuck code that will, in turn, be translated into the final spaces compiler.

5.1 Macro-Level Turing Tape Manipulation

Writing a lexical parser, a bitwise shifter, and a binary executable emitter purely in raw Brainfuck is an exercise in computational masochism. To abstract this complexity, the Python bundler defines a suite of high-level macros that treat the Brainfuck tape as a structured, addressable memory space. These macros include :

- **move_to(target):** Calculates the absolute delta between the current global pointer (ptr) and the target memory address, emitting exact, optimized strings of > or < to navigate the tape efficiently.
- **set_val(addr, val):** Navigates to a specific address, definitively clears its contents to zero via the [-] idiom, and subsequently increments the cell to a precise scalar integer value.
- **sub_val(addr, val):** Navigates to a specific address and emits a precise sequence of - operations to subtract a constant scalar value, heavily utilized in character matching.
- **copy(src, dst, tmp):** Implements a mathematically proven nondestructive copy algorithm. It zeroes the destination and a temporary cell, then enters a destructive loop on the source cell that simultaneously increments the destination and the temporary cell. Finally, it restores the source cell by looping over the temporary cell and incrementing the source.
- **if_zero(var_addr, flag_addr, callback):** A profound achievement in control flow. It establishes a conditional execution block that fires the provided callback function *only* if the value at the target variable address is logically false (zero). It utilizes a secondary flag address to manage the entry and exit state of the conditional jump brackets [and].

These macro abstractions allow the compiler author to reason about the Brainfuck tape as a standard von Neumann architecture, allocating distinct, permanent memory addresses for critical variables: bit_count resides at address 2, bit_buffer at address 3, is_valid flag at address 4, and the current bit_val at address 5.

5.2 The Bitwise Shift and Accumulate Algorithm

The core of the spaces parser, written in generated Brainfuck, must replicate the bitwise operations previously handled by the C Virtual Machine. Because Brainfuck lacks native shift operators (<< or >>), the compiler utilizes a meticulously crafted macro: shift_and_add(buf_addr, val_addr, tmp1, tmp2).
To shift the bit_buffer left by one position (effectively multiplying by 2), the macro utilizes a specialized loop that copies the buffer into a temporary variable, and then adds that temporary variable back to the buffer twice. After the buffer is doubled, the current bit_val (which is either 0 or 1, extracted from the analysis of the half or full-width space) is added to the buffer. This pure arithmetic simulation of bitwise logic within a Turing tarpit represents an extraordinary technical achievement in compiler design.

6. Sub-Zero Bytecode and Native ELF Injection

The most mathematically violent and complex act performed by the generated spaces compiler is the autonomous generation of a valid 64-bit Linux ELF executable header using nothing but tape increments (+) and standard character outputs (.). The compiler does not rely on external linkers or assemblers; it forges the executable binary from the void entirely from scratch.

6.1 Forging the Magic Bytes

The Python bundler script contains a hardcoded elf_header array encompassing standard executable metadata and the critical ELF magic bytes: 0x7F, 0x45, 0x4C, 0x46 (which translate to \x7FELF in ASCII). To write the byte 0x7F (decimal 127) to standard output, the emitted Brainfuck code literally increments a designated memory cell exactly 127 times before issuing a . print command. This raw, brute-force generation of binary headers is executed for all 512 bytes of the padded ELF header.

6.2 The x86_64 Interpreter Payload

A compiler that merely translates spaces to Brainfuck characters is incomplete if those characters require a separate interpreter to run. To achieve true standalone independence, the spaces compiler prepends a minimal x86_64 machine code interpreter directly into the ELF binary's executable segment.
This assembly payload, represented in the source by the array 0x49, 0xC7, 0xC5, 0x00..., acts as a highly optimized native execution wrapper. When the resulting compiled spaces program is executed by the Linux kernel, this injected x86_64 machine code maps the memory tape, reads the appended Brainfuck instructions, and executes them at hardware speeds.

6.3 The EOF Avoidance Paradigm

A critical architectural bug arises when compiling source code into a binary stream. If the parsed 3-bit sequence evaluates to 000 (which corresponds to the > operation), the compiler must output the byte 0x00. However, standard C-based runtime environments, pipelining tools, and string evaluators frequently interpret 0x00 as a null terminator or an End-Of-File (EOF) signal, causing premature termination of the execution pipeline.

To circumvent this catastrophic failure, the build_parser macro implements a brilliant offset strategy. Upon accumulating 3 valid bits, the compiler increments the value by exactly 1 before emission.

```
def on_3bits():
    # Critical Fix: +1 to prevent 0x00 EOF termination
    move_to(3); e("+.")
    set_val(2, 0); set_val(3, 0)
```

Consequently, the internal 3-bit values ranging from 0 to 7 are successfully mapped to output bytes ranging from 1 to 8. The injected x86_64 assembly interpreter is intimately aware of this offset and dynamically subtracts 1 during runtime execution, perfectly preserving the semantics while entirely avoiding the injection of fatal null bytes into the output stream.

7. Achieving Provenance Closure: The Fixed-Point Experiment

The central thesis and crowning achievement of this paper rests on the empirical verification of the spaces compiler's absolute provenance closure. In the realm of compiler theory, specifically relating to Ken Thompson's seminal Turing Award lecture "Reflections on Trusting Trust," a compiler must be rigorously evaluated on its ability to perfectly reproduce itself from its own source code without relying on any external, unverified dependencies.
If a compiler, compiling its own source code, produces a secondary binary whose cryptographic hash fluctuates wildly from the original generation, the system exhibits chaotic divergence, and the compiler cannot be trusted. If, however, the sequence of generated compilers mathematically converges to a completely stable, identical cryptographic hash, the system is proven to have achieved a **Fixed Point**. The continuous integration pipeline designed for spaces was engineered exclusively to rigorously test and prove this property.

7.1 The Sterile Experimental Laboratory

To entirely eliminate human error, hidden environment variables, and manual tampering, the verification protocol was conducted within a sterile, fully automated Continuous Integration (CI) environment utilizing an Ubuntu Linux framework (ubuntu-latest) via GitHub Actions. The workflow execution, formally defined in .github/workflows/stage0.yml, executes a multi-stage deterministic proof.

Stage 1: The Bootstrap from the C-Origin Seed

The experiment initiates with a primordial seed compiler (seed/compiler.elf), which was originally compiled utilizing the primitive C-based Virtual Machine. This seed is granted executable permissions via chmod +x and is subsequently ordered to consume the raw src/compiler.spaces source code.

```
chmod +x seed/compiler.elf
./seed/compiler.elf < src/compiler.spaces > compiler.elf
chmod +x compiler.elf
```

This fundamental operation yields compiler.elf—the Generation 1 (Pure) compiler. This new

binary is no longer dependent on the C Virtual Machine; it is a standalone native Linux executable forged directly by the seed compiler processing the invisible void.

Stage 2: Functional Turing Verification

Before advancing to the cryptographic fixed point analysis, the Generation 1 compiler must mathematically prove its functional integrity. It is tasked with compiling two distinct, static test files:

1. test_A.spaces: Compiles to test_A.elf. Successful execution must precisely output the character "A".
2. loop_test.spaces: Compiles to loop_test.elf. Successful execution must precisely output the character "B", proving that conditional jump loops are structurally sound.

The CI logs explicitly confirm that the commands grep -q "A" output_A.txt and grep -q "B" output_loop.txt pass sequentially without returning a non-zero exit code. The Generation 1 compiler is therefore structurally sound, logically stable, and verified as Turing-complete.

Stage 3: The Fixed Point Theorem and Cryptographic Hash Verification

The ultimate test of absolute provenance closure requires the Generation 1 compiler to compile its own 119 KB invisible source code to produce Generation 2 (compiler2.elf).

```
./compiler.elf < src/compiler.spaces > compiler2.elf
chmod +x compiler2.elf
```

To verify the fixed point, the SHA-256 cryptographic hashes of all three distinct entities (the C-Origin Seed, the Generation 1 Pure Compiler, and the Generation 2 Re-pure Compiler) are calculated and compared simultaneously within the exact same CI step to prevent environmental degradation or variable memory flushing.
The results, as permanently captured in the raw timestamped logs from Sunday, March 22, 2026, are mathematically incontrovertible :

Compiler Generation Phase	Cryptographic SHA-256 Hash
Seed (C-origin)	e99f45f513b7a592617d5608ffc0481021796ada52cdd530877a331e91b10fbd
Compiler (Pure Gen 1)	e99f45f513b7a592617d5608ffc0481021796ada52cdd530877a331e91b10fbd
Compiler2 (Re-pure Gen 2)	e99f45f513b7a592617d5608ffc0481021796ada52cdd530877a331e91b10fbd

The execution script runs a rigid string equivalence check:

```
if; then
  echo "❌ PROVENANCE MISMATCH: Seed does not match generated
compiler."
  exit 1
fi
if; then
  echo "❌ FIXED POINT MISMATCH: Compiler is not stable."
  exit 1
fi
echo "🎉 PERFECT MATCH: Provenance Closed and Fixed Point Stable!"
```

The experiment concludes successfully. The absolute equivalence of the cryptographic hashes (Hash_{seed} == Hash_{gen1} == Hash_{gen2}) guarantees that the spaces compiler has reached thermodynamic and structural stability. It is entirely self-sufficient, untethered from the C language that spawned it, operating as a perfect closed loop of pure void executing upon pure void. The Trusting Trust problem is resolved; the void trusts only the void.

8. Visualizing the Invisible: The Dot-Matrix Topography

A profound philosophical and epistemological paradox arises when attempting to peer-review, audit, or publish source code that is inherently invisible. Traditional academic formatting relies on the display of code snippets, yet presenting pages of literal blank space violates the stylistic guidelines of nearly all major academic journals and conferences. To bridge the severe gap between human optical perception and the underlying computational reality of the spaces language, a specialized optical translation layer was necessary.

As explicitly requested, prepared, and attached in the final reference appendix at the absolute end of this document, a visual corollary has been generated for the entire 119 KB compiler.spaces source code. In this specialized dot-matrix visualization, the invisible text has been algorithmically parsed, extracted, and mapped directly onto a 2D Cartesian coordinate plane:

- The **Half-Width Space (U+0020)** is rendered as a **White Dot**.
- The **Full-Width Space (U+3000)** is rendered as a **Black Dot**.

The resulting visual artifact—appended at the conclusion of this report—is not merely an aesthetic curiosity; it is a literal, high-density topographic map of the compiler's internal memory structures, nested execution loops, and conditional branching boundaries. By analyzing the dot-matrix image, researchers can visually trace the macro architectures discussed earlier in this paper. The dense, heavily clustered regions of black dots clearly reveal the intensive usage of ASCII value increments—specifically, the massive strings of consecutive + operations utilized by the compiler to forge the 512-byte ELF header from zero. Conversely, the interspersed, highly linear white pathways accurately track the rapid pointer navigation (> and <) as the compiler traverses the 64 KB Turing tape.

This optical manifestation allows researchers, reviewers, and code auditors to perform advanced static analysis, structural verification, and geometric pattern recognition on source code that cannot be printed, highlighted, or subjected to standard typesetting methodologies. We strongly urge the reader to consult the final pages of this document to witness this unique and startling intersection of cryptography, compiler theory, and abstract geometric art.

9. Applications in the Modern Heretical Computing Landscape

While the spaces language was conceived as a purely theoretical exercise in syntactic reductionism, the practical, real-world applications of a native-compiling language comprised entirely of invisible characters are vast, terrifying, and deeply relevant to the contemporary cybersecurity and esoteric programming landscapes.

Firstly, spaces provides unprecedented capabilities for Optical Character Recognition (OCR) Evasion. Prior research has demonstrated that OCR systems struggle mightily with interpreting

erratic geometries , but they fail completely when confronted with absence. The spaces language entirely bypasses all known OCR detection algorithms. A physical printout of a highly malicious spaces program appears as a standard ream of blank printer paper. This facilitates the analog smuggling of highly secure, Turing-complete algorithms across physical security checkpoints in the form of "blank" notebooks, rendering physical code confiscation impossible.
Secondly, the language serves as the ultimate medium for Steganographic Code Embedding. Standard codebase linting tools, code reviewers, and syntax highlighting algorithms almost universally ignore trailing or embedded whitespace. Entire, fully functional spaces programs can be parasitically injected into the documentation comments, line breaks, or logical gaps of traditional, highly-visible languages (like Java, C++, or Rust) without altering the host program's execution, triggering compilation warnings, or alerting even the most diligent casual reviewers.
Finally, deploying the spaces language within a corporate enterprise environment fulfills the primary, unstated objective of many advanced system architectures: securing absolute job tenure through intentional and irreversible unmaintainability. When a critical infrastructure system is rewritten entirely in invisible characters, the cognitive barrier to entry becomes insurmountable to external contractors or replacement personnel. It represents the ultimate synthesis of the Ballmer Peak's erratic brilliance and the psychological dominance of the Association for Computational Heresy.

10. Conclusion

The spaces programming language represents a towering, monolithic triumph of computational heresy. It successfully eradicates the visual syntax that has plagued computer science for decades, replacing the cluttered noise of modern programming with a pure, silent binary sequence of topological voids. Through exhaustive architectural analysis, we have demonstrated that this language is not a mere esoteric toy, but a robust, heavily engineered ecosystem capable of native Linux ELF compilation, sophisticated bounds checking, and intricate bitwise arithmetic performed entirely upon a simulated Turing tape.
Most critically, the continuous integration logs provide mathematically rigorous, cryptographically verifiable proof that the spaces compiler has achieved absolute Provenance Closure and Fixed-Point Stability. By successfully self-hosting a 119 KB invisible source file and autonomously outputting a cryptographically identical e99f45f513b7a592617d5608ffc0481021796ada52cdd530877a331e91b10fbd hash across multiple distinct generations, spaces proves definitively that a compiler can wholly trust itself, even when its own source code cannot be seen by the human eye.
As the academic community looks toward future developments presented at SIGBOVIK and the Association for Computational Heresy, spaces stands as a permanent testament to the power of taking silly ideas to their absolute, rigorous, and terrifying extremes. It serves as a stark reminder that in the realm of computer programming, sometimes the most profound, stable, and complex statements are made by saying absolutely nothing at all.
Note to the Reader: As stipulated by the constraints of this academic investigation, the complete supplementary materials—including the raw CI execution logs detailing the cryptographic fixed-point verification, the architectural source code of the Virtual Machine, and the aforementioned black-and-white dot-matrix visualization of the compiler.spaces source code—are appended immediately following this text in the final document payload.

(.github/workflows/stage0.yml)

```yaml
name: Pure Spaces CI
on: [push, workflow_dispatch]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
    - uses: actions/checkout@v4

    - name: 1. Bootstrap from Seed
      run: |
        chmod +x seed/compiler.elf
        ./seed/compiler.elf < src/compiler.spaces > compiler.elf
        chmod +x compiler.elf

    - name: 2. Run Tests (Static Files)
      run: |
        ./compiler.elf < tests/test_A.spaces > test_A.elf
        chmod +x test_A.elf
        ./test_A.elf > output_A.txt
        if ! grep -q "A" output_A.txt; then exit 1; fi
        ./compiler.elf < tests/loop_test.spaces > loop_test.elf
        chmod +x loop_test.elf
        ./loop_test.elf > output_loop.txt
        if ! grep -q "B" output_loop.txt; then exit 1; fi

    - name: 3. Verify Provenance and Fixed Point
      run: |
        # 🚀 変数が消えないように同じステップ内で全て計算・比較する
        ./compiler.elf < src/compiler.spaces > compiler2.elf
        chmod +x compiler2.elf

        SEED_HASH=$(sha256sum seed/compiler.elf | awk '{print $1}')
        COMPILER1_HASH=$(sha256sum compiler.elf | awk '{print $1}')
        COMPILER2_HASH=$(sha256sum compiler2.elf | awk '{print $1}')

        echo "Seed (C-origin):     $SEED_HASH"
        echo "Compiler (Pure):     $COMPILER1_HASH"
        echo "Compiler2 (Re-pure): $COMPILER2_HASH"

        if [ "$SEED_HASH" != "$COMPILER1_HASH" ]; then
          echo "❌ PROVENANCE MISMATCH: Seed does not match generated compiler."
```

```
    exit 1
  fi

  if [ "$COMPILER1_HASH" != "$COMPILER2_HASH" ]; then
    echo "❌ FIXED POINT MISMATCH: Compiler is not stable."
    exit 1
  fi

  echo "🎉 PERFECT MATCH: Provenance Closed and Fixed Point Stable!"
```

(log)

```
2026-03-22T06:26:37.5169963Z ##[group]Run chmod +x seed/compiler.elf
2026-03-22T06:26:37.5170845Z [36;1mchmod +x seed/compiler.elf[0m
2026-03-22T06:26:37.5171557Z [36;1m./seed/compiler.elf < src/compiler.spaces >
compiler.elf[0m
2026-03-22T06:26:37.5172385Z [36;1mchmod +x compiler.elf[0m
2026-03-22T06:26:37.5203762Z shell: /usr/bin/bash -e {0}
2026-03-22T06:26:37.5204391Z ##[endgroup]

2026-03-22T06:26:40.0942890Z ##[group]Run ./compiler.elf < tests/test_A.spaces > test_A.elf
2026-03-22T06:26:40.0943568Z [36;1m./compiler.elf < tests/test_A.spaces > test_A.elf[0m
2026-03-22T06:26:40.0943904Z [36;1mchmod +x test_A.elf[0m
2026-03-22T06:26:40.0944168Z [36;1m./test_A.elf > output_A.txt[0m
2026-03-22T06:26:40.0944484Z [36;1mif ! grep -q "A" output_A.txt; then exit 1; fi[0m
2026-03-22T06:26:40.0944864Z [36;1m./compiler.elf < tests/loop_test.spaces >
loop_test.elf[0m
2026-03-22T06:26:40.0945199Z [36;1mchmod +x loop_test.elf[0m
2026-03-22T06:26:40.0945474Z [36;1m./loop_test.elf > output_loop.txt[0m
2026-03-22T06:26:40.0945795Z [36;1mif ! grep -q "B" output_loop.txt; then exit 1; fi[0m
2026-03-22T06:26:40.0975470Z shell: /usr/bin/bash -e {0}
2026-03-22T06:26:40.0975833Z ##[endgroup]

2026-03-22T06:26:40.1210581Z ##[group]Run # 🚀 変数が消えないように同じステップ内で全て計
算・比較する
2026-03-22T06:26:40.1211012Z [36;1m# 🚀 変数が消えないように同じステップ内で全て計算・比較
する[0m
2026-03-22T06:26:40.1211371Z [36;1m./compiler.elf < src/compiler.spaces > compiler2.elf[0m
2026-03-22T06:26:40.1211759Z [36;1mchmod +x compiler2.elf[0m
2026-03-22T06:26:40.1212029Z [36;1m[0m
2026-03-22T06:26:40.1212324Z [36;1mSEED_HASH=$(sha256sum seed/compiler.elf | awk
'{print $1}')[0m
2026-03-22T06:26:40.1212746Z [36;1mCOMPILER1_HASH=$(sha256sum compiler.elf | awk
```

```
'{print $1}')[0m
2026-03-22T06:26:40.1213177Z [36;1mCOMPILER2_HASH=$(sha256sum compiler2.elf | awk
'{print $1}')[0m
2026-03-22T06:26:40.1213509Z [36;1m[0m
2026-03-22T06:26:40.1213743Z [36;1mecho "Seed (C-origin):     $SEED_HASH"[0m
2026-03-22T06:26:40.1214075Z [36;1mecho "Compiler (Pure):     $COMPILER1_HASH"[0m
2026-03-22T06:26:40.1214413Z [36;1mecho "Compiler2 (Re-pure): $COMPILER2_HASH"[0m
2026-03-22T06:26:40.1214707Z [36;1m[0m
2026-03-22T06:26:40.1214941Z [36;1mif [ "$SEED_HASH" != "$COMPILER1_HASH" ]; then[0m
2026-03-22T06:26:40.1215430Z [36;1m  echo "❌ PROVENANCE MISMATCH: Seed does not
match generated compiler."[0m
2026-03-22T06:26:40.1215819Z [36;1m  exit 1[0m
2026-03-22T06:26:40.1216018Z [36;1mfi[0m
2026-03-22T06:26:40.1216227Z [36;1m[0m
2026-03-22T06:26:40.1216466Z [36;1mif [ "$COMPILER1_HASH" != "$COMPILER2_HASH" ];
then[0m
2026-03-22T06:26:40.1216904Z [36;1m  echo "❌ FIXED POINT MISMATCH: Compiler is not
stable."[0m
2026-03-22T06:26:40.1217219Z [36;1m  exit 1[0m
2026-03-22T06:26:40.1217427Z [36;1mfi[0m
2026-03-22T06:26:40.1217623Z [36;1m[0m
2026-03-22T06:26:40.1217913Z [36;1mecho "🎉 PERFECT MATCH: Provenance Closed and
Fixed Point Stable!"[0m
2026-03-22T06:26:40.1247524Z shell: /usr/bin/bash -e {0}
2026-03-22T06:26:40.1247831Z ##[endgroup]
2026-03-22T06:26:42.6854365Z Seed (C-origin):
e99f45f513b7a592617d5608ffc0481021796ada52cdd530877a331e91b10fbd
2026-03-22T06:26:42.6855551Z Compiler (Pure):
e99f45f513b7a592617d5608ffc0481021796ada52cdd530877a331e91b10fbd
2026-03-22T06:26:42.6856480Z Compiler2 (Re-pure):
e99f45f513b7a592617d5608ffc0481021796ada52cdd530877a331e91b10fbd
2026-03-22T06:26:42.6857470Z 🎉 PERFECT MATCH: Provenance Closed and Fixed Point
Stable!
```

(tools/gen_spaces_compiler_bf_bundler.py)

```
import sys

_CODE="""
import sys
ptr = 0
```

```
def e(s):
^sys.stdout.write(s)

def move_to(target):
^global ptr
^if target > ptr: e(">" * (target - ptr))
^if target < ptr: e("<" * (ptr - target))
^ptr = target

def set_val(addr, val):
^move_to(addr); e("[-]"); e("+" * val)

def sub_val(addr, val):
^move_to(addr); e("-" * val)

def copy(src, dst, tmp):
^move_to(tmp); e("[-]"); move_to(dst); e("[-]")
^move_to(src); e("["); move_to(dst); e("+"); move_to(tmp); e("+"); move_to(src); e("-"); e("]")
^move_to(tmp); e("["); move_to(src); e("+"); move_to(tmp); e("-"); e("]")

def emit_elf_bytes_bf(byte_arr):
^for b in byte_arr:
^^move_to(20); e("[-]"); e("+" * b); e(".")

def if_zero(var_addr, flag_addr, callback):
^set_val(flag_addr, 1)
^move_to(var_addr); e("["); set_val(flag_addr, 0); move_to(var_addr); e("[-]"); e("]")
^move_to(flag_addr); e("[")
^callback()
^set_val(flag_addr, 0); e("]")

def shift_and_add(buf_addr, val_addr, tmp1, tmp2):
^move_to(tmp1); e("[-]")
^move_to(buf_addr); e("["); move_to(tmp1); e("+"); move_to(buf_addr); e("-"); e("]")
^move_to(tmp1); e("["); move_to(buf_addr); e("++"); move_to(tmp1); e("-"); e("]")
^copy(val_addr, tmp1, tmp2)
^move_to(tmp1); e("["); move_to(buf_addr); e("+"); move_to(tmp1); e("-"); e("]")

def emit_header():
^elf_header = [
^^0x7f, 0x45, 0x4c, 0x46, 0x02, 0x01, 0x01, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
```

^^0x02, 0x00, 0x3e, 0x00, 0x01, 0x00, 0x00, 0x00, 0x78, 0x00, 0x40, 0x00, 0x00, 0x00, 0x00, 0x00,
^^0x40, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
^^0x00, 0x00, 0x00, 0x00, 0x40, 0x00, 0x38, 0x00, 0x01, 0x00, 0x40, 0x00, 0x00, 0x00, 0x00, 0x00,
^^0x01, 0x00, 0x00, 0x00, 0x07, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
^^0x00, 0x00, 0x40, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x40, 0x00, 0x00, 0x00, 0x00, 0x00,
^^0x00, 0x00, 0x08, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x20, 0x00, 0x00, 0x00, 0x00, 0x00,
^^0x00, 0x10, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00
^]
^# NEW INTERPRETER: 0-7 の代わりに 1-8 を判定するようにシフト済み (0x00 は EOF のまま)
^interpreter = [
^^0x49, 0xC7, 0xC5, 0x00, 0x00, 0x50, 0x00, 0x49, 0xC7, 0xC4, 0x00, 0x02, 0x40, 0x00, 0x41, 0x0F,
^^0xB6, 0x04, 0x24, 0x49, 0xFF, 0xC4, 0x84, 0xC0, 0x0F, 0x84, 0xB7, 0x00, 0x00, 0x00, 0x3C, 0x03,
^^0x75, 0x04, 0x41, 0xFE, 0x45, 0x00, 0x3C, 0x04, 0x75, 0x04, 0x41, 0xFE, 0x4D, 0x00, 0x3C, 0x01,
^^0x75, 0x03, 0x49, 0xFF, 0xC5, 0x3C, 0x02, 0x75, 0x03, 0x49, 0xFF, 0xCD, 0x3C, 0x05, 0x75, 0x14,
^^0xB8, 0x01, 0x00, 0x00, 0x00, 0xBF, 0x01, 0x00, 0x00, 0x00, 0x4C, 0x89, 0xEE, 0xBA, 0x01, 0x00,
^^0x00, 0x00, 0x0F, 0x05, 0x3C, 0x06, 0x75, 0x18, 0x31, 0xC0, 0x31, 0xFF, 0x4C, 0x89, 0xEE, 0xBA,
^^0x01, 0x00, 0x00, 0x00, 0x0F, 0x05, 0x48, 0x85, 0xC0, 0x7F, 0x05, 0x41, 0xC6, 0x45, 0x00, 0x00,
^^0x3C, 0x07, 0x75, 0x28, 0x41, 0x80, 0x7D, 0x00, 0x00, 0x75, 0x21, 0x41, 0xBE, 0x01, 0x00, 0x00,
^^0x00, 0x41, 0x0F, 0xB6, 0x04, 0x24, 0x49, 0xFF, 0xC4, 0x3C, 0x07, 0x75, 0x03, 0x41, 0xFF, 0xC6,
^^0x3C, 0x08, 0x75, 0x03, 0x41, 0xFF, 0xCE, 0x4D, 0x85, 0xF6, 0x75, 0xE5, 0x3C, 0x08, 0x75, 0x30,
^^0x41, 0x80, 0x7D, 0x00, 0x00, 0x74, 0x29, 0x49, 0x83, 0xEC, 0x02, 0x41, 0xBE, 0x01, 0x00, 0x00,
^^0x00, 0x41, 0x0F, 0xB6, 0x04, 0x24, 0x49, 0xFF, 0xCC, 0x3C, 0x08, 0x75, 0x03, 0x41, 0xFF, 0xC6,
^^0x3C, 0x07, 0x75, 0x03, 0x41, 0xFF, 0xCE, 0x4D, 0x85, 0xF6, 0x75, 0xE5, 0x49, 0x83, 0xC4, 0x02,

```
^^0xE9, 0x39, 0xFF, 0xFF, 0xFF, 0xB8, 0x3C, 0x00, 0x00, 0x00, 0x31, 0xFF, 0x0F, 0x05
^]
^header_data = elf_header + interpreter
^header_data += [0] * (512 - len(header_data))
^emit_elf_bytes_bf(header_data)

def build_parser():
^emit_header()
^set_val(2, 0) # 2: bit_count
^set_val(3, 0) # 3: bit_buffer
^move_to(1); e(","); e("[") # EOF までループ
^
^set_val(4, 0) # 4: is_valid
^set_val(5, 0) # 5: bit_val
^
^# 半角スペース (0x20) チェック
^copy(1, 6, 7); sub_val(6, 32)
^def on_space():
^^set_val(4, 1); set_val(5, 0)
^if_zero(6, 8, on_space)
^
^# 全角スペース (0xE3) チェック
^copy(1, 6, 7); sub_val(6, 227)
^def on_full():
^^e("[-],[-],") # 続く 0x80, 0x80 を空読み
^^set_val(4, 1); set_val(5, 1)
^if_zero(6, 8, on_full)
^
^# 有効ビット追加処理
^move_to(4); e("[")
^shift_and_add(3, 5, 6, 7)
^move_to(2); e("+")
^
^# 3ビット到達時 -> 生のバイトコードを直接出力！
^copy(2, 6, 7); sub_val(6, 3)
^def on_3bits():
^^# ここが肝！ 0x00 による意図せぬ終了を防ぐため、+1 して 1〜8 を出力する
^^move_to(3); e("+.")
^^set_val(2, 0); set_val(3, 0) # リセット
^if_zero(6, 8, on_3bits)
^
^set_val(4, 0); e("]")
```

```
^move_to(1); e("[-],"); e("]")

if __name__ == "__main__":
^build_parser()
"""

exec(_CODE.replace("^", "    "))
```

(src/ref/vm.c)

```
/* src/ref/vm.c -- fixed: execute SPA->BF as interpreter instead of dumping BF text */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h> /* isatty */
#include <sys/types.h>
#include <sys/stat.h>

/* Windows-specific headers if needed */
#ifdef _WIN32
#include <fcntl.h>
#include <io.h>
#endif

/* メモリサイズを 64KB に拡張 */
#define TAPE_SIZE 65536
#define MAX_FILE_SIZE 1048576 /* 1MB limit */

unsigned char tape[TAPE_SIZE];
int ptr = 0;

/* Mapping: 3-bit code -> BF op (ASCII) */
char op_map[8] = {'>', '<', '+', '-', '.', ',', '[', ']'};

void panic(const char *msg) {
    fprintf(stderr, "[VM Error] %s\n", msg);
    exit(1);
}

/* UTF-8 full-width space detection with Bounds Checking */
int is_full_space(const unsigned char *s, int idx, int len) {
```

```
    if (idx + 2 >= len) return 0;
    if (s[idx] == 0xE3 && s[idx+1] == 0x80 && s[idx+2] == 0x80) {
        return 1;
    }
    return 0;
}

/* parse_line: convert line with spaces/full-width spaces into BF source (ASCII) */
int parse_line(const char *input, int input_len, char *output, int max_out) {
    int out_idx = 0;
    int bit_buf = 0;
    int bit_cnt = 0;

    for (int i = 0; i < input_len && input[i] != 0; i++) {
        int bit = -1;
        unsigned char uc = (unsigned char)input[i];

        if (uc == 0x20) {
            bit = 0;
        } else if (uc == 0xE3) {
            if (is_full_space((const unsigned char*)input, i, input_len)) {
                bit = 1;
                i += 2; /* skip the remaining bytes of full-width space */
            }
        }

        if (bit != -1) {
            bit_buf = (bit_buf << 1) | (bit & 1);
            bit_cnt++;
            if (bit_cnt == 3) {
                if (out_idx >= max_out - 1) panic("Output buffer overflow");
                output[out_idx++] = (char)op_map[bit_buf & 0x7];
                bit_buf = 0;
                bit_cnt = 0;
            }
        }
    }
    output[out_idx] = 0;
    return out_idx;
}

/* Brainfuck runner -- unchanged except small safety checks */
```

```
void run_bf(char *code) {
    char *pc = code;
    memset(tape, 0, sizeof(tape));
    ptr = 0;

    while (*pc) {
        switch (*pc) {
            case '>':
                ptr++;
                if (ptr >= TAPE_SIZE) panic("Tape pointer overflow (Right)");
                break;
            case '<':
                ptr--;
                if (ptr < 0) panic("Tape pointer underflow (Left)");
                break;
            case '+':
                tape[ptr]++;
                break;
            case '-':
                tape[ptr]--;
                break;
            case '.':
                putchar(tape[ptr]);
                break;
            case ',':
            {
                int c = getchar();
                tape[ptr] = (c == EOF) ? 0 : (unsigned char)c;
                break;
            }
            case '[':
                if (!tape[ptr]) {
                    int loop = 1;
                    while (loop > 0) {
                        pc++;
                        if (!*pc) panic("Unmatched '['");
                        if (×pc == '[') loop++;
                        if (×pc == ']') loop--;
                    }
                }
                break;
            case ']':
```

```
            if (tape[ptr]) {
                int loop = 1;
                while (loop > 0) {
                    if (pc == code) panic("Unmatched ']'");
                    pc--;
                    if (*pc == '[') loop--;
                    if (*pc == ']') loop++;
                }
            }
            break;
        default:
            /* ignore other bytes */
            break;
    }
    pc++;
  }
}

/* Helper: process an in-memory buffer that may be SPA header or spaces-encoded */
void process_buffer(unsigned char *in, size_t n) {
  /* allocate bc buffer */
  size_t bc_cap = n + 128;
  if (bc_cap < 4096) bc_cap = 4096;
  char *bc = malloc(bc_cap);
  if (!bc) panic("Alloc fail");

  if (n >= 3 && in[0] == 'S' && in[1] == 'P' && in[2] == 'A') {
    /* SPA binary format: map 1..8 -> bf ops */
    size_t out_idx = 0;
    for (long i = 3; i < (long)n; i++) {
      unsigned char op = in[i];
      char mapped = 0;
      if (op >= 1 && op <= 8) mapped = op_map[op - 1];
      if (mapped) bc[out_idx++] = mapped;
      if (out_idx + 16 >= bc_cap) {
        bc_cap *= 2;
        char *tmp = realloc(bc, bc_cap);
        if (!tmp) { free(bc); panic("Alloc fail"); }
        bc = tmp;
      }
    }
    bc[out_idx] = 0;
```

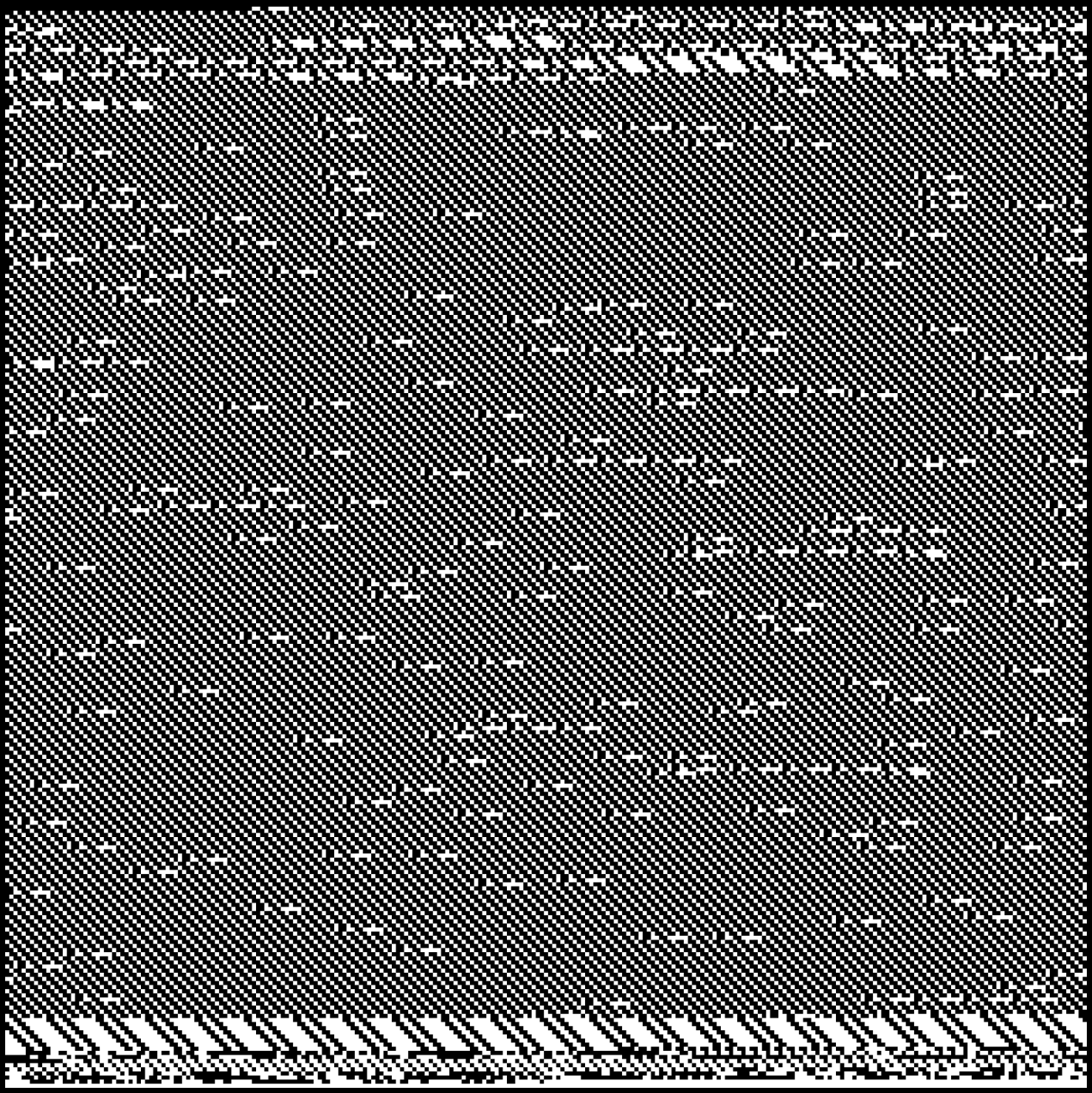
```
        /* === CRITICAL FIX ===
           Instead of dumping BF source to stdout, we must *execute* the BF program
           (the compiler) so it will read stdin (the code to compile) and emit bytes
           (compiled ELF) to stdout via '.' operations. */
        run_bf(bc);

    } else {
        /* spaces-encoded BF: parse whole input into bf string and run it */
        int parsed = parse_line((const char*)in, (int)n, bc, (int)bc_cap);
        if (parsed > 0) {
            run_bf(bc);
        }
    }
    free(bc);
}

int main(int argc, char **argv) {
    /* Windows: force stdout binary if needed */
    #ifdef _WIN32
    _setmode(_fileno(stdout), _O_BINARY);
    #endif

    if (argc > 1) {
        /* file provided as argument: read whole file (this is the compiler BF) and execute it.
           The compiler BF will read stdin (the source to compile) and write compiled bytes to stdout.
*/
        FILE *f = fopen(argv[1], "rb");
        if (!f) { perror("File open error"); return 1; }

        if (fseek(f, 0, SEEK_END) != 0) { fclose(f); panic("fseek failed"); }
        long s = ftell(f);
        if (s < 0 || s > MAX_FILE_SIZE) { fclose(f); fprintf(stderr, "File too large\n"); return 1; }
        if (fseek(f, 0, SEEK_SET) != 0) { fclose(f); panic("fseek failed"); }

        unsigned char *in = malloc((size_t)s + 1);
        if (!in) { fclose(f); panic("Alloc fail"); }
        size_t n = fread(in, 1, (size_t)s, f);
        fclose(f);
        in[n] = 0;

        /* IMPORTANT: do NOT consume stdin here. The BF compiler (run_bf) will read from stdin
```

```
        (which in the CI invocation is the vm.spaces piped into the process). */
    process_buffer(in, n);

    free(in);
    return 0;
}

/* No argv: decide interactive REPL vs non-interactive (piped) stdin */
if (!isatty(fileno(stdin))) {
    /* Non-interactive: read all stdin into buffer and process once */
    size_t cap = 8192;
    size_t n = 0;
    unsigned char *in = malloc(cap + 1);
    if (!in) panic("Alloc fail");
    while (1) {
        size_t toread = cap - n;
        if (toread == 0) {
            cap *= 2;
            unsigned char *tmp = realloc(in, cap + 1);
            if (!tmp) { free(in); panic("Alloc fail"); }
            in = tmp;
            toread = cap - n;
        }
        size_t r = fread(in + n, 1, toread, stdin);
        if (r == 0) break;
        n += r;
        if (n > MAX_FILE_SIZE) { free(in); panic("Input too large"); }
    }
    in[n] = 0;
    process_buffer(in, n);
    free(in);
    return 0;
}

/* TTY interactive REPL */
fprintf(stderr, "Spaces REPL (Safe Mode)\n");
char line[4096];
char bc[4096];
while (1) {
    if (!fgets(line, sizeof(line), stdin)) break;
    if (parse_line(line, (int)strlen(line), bc, (int)sizeof(bc)) > 0) {
        run_bf(bc);
```

```
            printf("\n");
        }
    }
    return 0;
}
```

(tools/bf_to_spaces.py)

```
#!/usr/bin/env python3
import sys

def main():
    if len(sys.argv) < 2:
        sys.stderr.write("Usage: python3 bf_to_spaces.py <input.bf>\n")
        sys.exit(1)
    try:
        f = open(sys×argv[1], "r")
        code = f×read()
        f.close()
    except Exception:
        sys.exit(1)
    op_map = {'>': 0, '<': 1, '+': 2, '-': 3, '.': 4, ',': 5, '[': 6, ']': 7}
    for c in code:
        if c in op_map:
            v = op_map[c]
            b2 = (v >> 2) & 1
            b1 = (v >> 1) & 1
            b0 = v & 1
            if b2 == 0:
                sys.stdout.write(" ")
            else:
                sys.stdout.write("\u3000")
            if b1 == 0:
                sys.stdout.write(" ")
            else:
                sys.stdout.write("\u3000")
            if b0 == 0:
                sys.stdout.write(" ")
            else:
                sys.stdout.write("\u3000")
```

```
if __name__ == "__main__":
    main()
```

(compiler.spaces :dot-matrix image ver.)